

**SCAS002****PROJECT REPORT**

HUMAN ACTIVITY RECOGNITION

Using Deep Learning and 3D ResNet-18

Machine Learning Project Report

A video-based system that analyzes human activities and predicts the activity using a trained deep learning model.

Name	Registration No.
Sanjana Kumari	12514414

Github repository link:-

<https://github.com/sanjanaTech-1/human-activity-recognition>

A complete end-to-end application combining deep learning, computer vision, Flask backend, and a web interface.

Input: Video → Frame Sampling: 16 Frames → Model: 3D ResNet-18 → Output: Activity + Confidence Score

PROJECT SNAPSHOT

Abstract

Human Activity Recognition is a computer vision and deep learning problem in which a system identifies the activity being performed in a video. In this project, a video is processed into a fixed number of frames and passed through a 3D ResNet-18 deep learning model. The model learns both visual information from individual frames and movement information across multiple frames. The trained model is connected to a Flask backend and a web interface where users can upload a video and receive a predicted activity along with a confidence score and additional analysis details.

1. Introduction

Videos contain much more information than a single image because they show changes and movement over time. Human Activity Recognition aims to automatically understand these movements and classify the activity taking place.

The purpose of this project is to build a complete working system that combines deep learning with a web application. The user uploads a video, the system processes selected frames, the trained model analyzes them, and the final prediction is displayed on the website.

2. Problem Statement

Manually analyzing a large number of videos to identify human activities can be time-consuming. A computer-based system is therefore required to automatically analyze video content and recognize the activity being performed.

The main challenge is that a video contains both spatial information, such as the appearance and position of a person, and temporal information, such as movement and changes between frames.

3. Objectives

- Develop a deep learning model for recognizing human activities from video.
- Preprocess video data into a suitable format for deep learning.
- Learn both spatial and temporal patterns from video frames.
- Classify videos into the selected activity classes.
- Connect the trained model with a Flask backend and web interface.
- Display the predicted activity and confidence score to the user.

4. Dataset

The project uses the UCF101 Human Action Recognition dataset. UCF101 is a widely used video dataset containing videos of different human activities.

For the current project, 5 activity classes were selected. Approximately 100 videos were used per class, resulting in approximately 500 videos in total. The data was divided into training, validation, and testing sets.

Dataset Component	Description
Dataset	UCF101 Human Action Recognition Dataset
Selected classes	5 activity classes
Videos per class	Approximately 100
Total videos	Approximately 500
Training data	Approximately 80%
Validation data	Approximately 10%
Testing data	Approximately 10%

5. Features Used in the Project

Unlike traditional machine learning projects where features are manually selected from a table, this project uses video data. The deep learning model automatically learns useful features from the video frames.

5.1 Spatial Features

Spatial features describe what is visible in a frame. These may include body posture, shape, objects, appearance, and the position of the person.

5.2 Temporal Features

Temporal features describe how the content changes over time. These include movement, direction, speed patterns, and changes in body position between frames.

The 3D convolutional network is useful because it can learn spatial and temporal information together.

6. Video Preprocessing

Raw videos cannot be directly given to the model in their original form because videos can have different lengths, resolutions, and formats. Therefore, preprocessing is performed.

- A video is selected as input.
- 16 representative frames are extracted.
- Frames are converted from BGR to RGB because OpenCV uses BGR by default.
- Each frame is resized to 112×112 pixels.
- Pixel values are normalized.
- The frames are converted into a PyTorch tensor.

The final input can be represented as $3 \times 16 \times 112 \times 112$, representing RGB channels, frames, height, and width.

7. Deep Learning Model: 3D ResNet-18

The main model used in the project is 3D ResNet-18. A 3D convolutional neural network is suitable for video because it processes spatial information and temporal information together.

ResNet uses residual or skip connections. These connections help information flow through deeper neural networks and make the training process more effective.

The final classification layer was adapted to produce predictions for the 5 selected activity classes.

8. Model Training

During training, the model receives preprocessed video frames and produces a prediction. The prediction is compared with the correct class label. The error is calculated using the loss function, and the model weights are updated using backpropagation and the optimizer.

Parameter	Value / Method
Model	3D ResNet-18
Loss function	Cross Entropy Loss
Optimizer	AdamW
Learning rate	0.0001
Batch size	2
Training epochs	10
Learning-rate scheduler	ReduceLROnPlateau

9. Training Workflow

- Input video frames are prepared.
- The 3D ResNet-18 model processes the frames.
- The model produces a prediction.
- The prediction is compared with the actual label.
- Cross Entropy Loss calculates the error.
- Backpropagation calculates gradients.
- AdamW updates the model weights.
- The process repeats over multiple epochs.

10. Model Evaluation

The model is evaluated using standard classification metrics.

- Accuracy: The percentage of correctly classified samples.
- Precision: How many predicted samples of a class were actually correct.
- Recall: How many actual samples of a class were correctly identified.
- F1-score: A balance between precision and recall.
- Confusion Matrix: A table that compares actual classes with predicted classes.

11. Prediction and Confidence Score

When a new video is uploaded, the model produces output scores for the different classes. These scores are converted into probabilities using Softmax. The class with the highest probability is selected as the final prediction.

The confidence score represents how strongly the model supports its selected prediction. Confidence should not be confused with overall model accuracy.

12. System Architecture

The project contains three main layers:

- Frontend: HTML, CSS, and JavaScript provide the user interface.
- Backend: Flask receives the uploaded video and connects the website with the machine learning model.
- Machine Learning Model: PyTorch and 3D ResNet-18 process the video and predict the activity.

Overall flow: User Uploads Video → Flask Backend → Video Preprocessing → 3D ResNet-18 → Prediction → Confidence Score → Website Result.

13. Working Prototype

The trained model is integrated into a working web application. The user can upload an activity video and click the Analyze Activity button. The backend processes the video and sends it to the trained model.

The interface displays the detected activity, confidence score, top predictions, video duration, number of frames analyzed, frame rate, processing time, and an analysis summary.

14. Advantages

- Automatically analyzes videos instead of requiring manual inspection.
- Uses deep learning to learn visual and movement patterns.
- Provides a user-friendly web interface.
- Displays both the predicted activity and confidence information.
- Can be extended to additional activity classes.

15. Limitations

- The current model is trained on only 5 selected activity classes.
- The model may not reliably recognize activities that were not present during training.
- The current dataset size is limited compared with the full UCF101 dataset.
- Video deep learning can require significant computational resources.
- Prediction quality depends on the quality and representativeness of the training data.

16. Future Scope

- Add more activity classes and more training videos.
- Use transfer learning with stronger pretrained video models.
- Apply additional data augmentation techniques.
- Explore transformer-based video recognition models.
- Improve deployment using GPU or cloud infrastructure.
- Add real-time camera-based activity recognition.

8.2 Role of the Optimizer

The optimizer updates the model's internal weights based on the error calculated during training. AdamW was used to help the model learn efficiently while also providing weight-decay regularization.

8.1 Role of the Loss Function

The loss function measures the difference between the actual activity label and the model's prediction. During training, the model tries to reduce this error. Cross Entropy Loss is suitable because the project is a multi-class classification problem.

7.1 Why 3D Convolution Is Suitable for Video

A normal 2D convolution mainly analyzes spatial patterns in individual images. A 3D convolution extends this idea across the time dimension. Therefore, it can learn patterns from a sequence of frames and capture movement information, which is important for activity recognition.

10.2 Confusion Matrix Interpretation

The diagonal values of a confusion matrix generally represent correct predictions, while values outside the diagonal represent cases where the model predicted a different class from the actual class. This helps identify activities that may look visually or behaviorally similar.

10.1 Why Accuracy Is Not Enough

Accuracy alone may not provide a complete understanding of a classification model. If some activity classes are easier to recognize than others, precision, recall, F1-score, and the confusion matrix provide additional information about the behavior of the model.

13.5 Importance of the Working Prototype

The working prototype demonstrates the complete path from user input to machine learning output. Therefore, the project is not limited to model training. It also demonstrates how a trained model can be integrated into a practical software application.

13.4 Result Display

The final result is returned to the frontend and displayed in a clear format. The interface shows the detected activity, confidence score, top predictions, number of frames analyzed, video duration, frame rate, processing time, and an analysis summary.

13.3 Model Prediction

The model processes the selected frames and generates prediction scores for the available activity classes. The scores are converted into probabilities, and the class with the highest probability is selected as the primary predicted activity.

13.2 Backend Processing

After the user clicks Analyze Activity, the video is sent from the frontend to the Flask backend. The backend receives the uploaded file, prepares it for analysis, and passes it to the machine learning pipeline. This backend layer acts as the connection between the website and the trained deep learning model.

13.1 User Interaction

The user interacts with the system through a simple web interface. The user selects a video file from the device and uploads it. After the video is selected, the interface shows the uploaded video and provides an Analyze Activity button. This makes the system easy to use even for a user who does not have technical knowledge.

The final outcome is a complete prototype that demonstrates the full machine learning lifecycle: dataset preparation, preprocessing, model training, evaluation, inference, backend integration, and frontend result presentation.

14.10 Project Outcome

The current system can be extended by improving the model, adding more classes, and deploying the backend on a suitable server. GPU-based inference can reduce processing time for larger videos or multiple users.

14.9 Scalability

In a production-level version, uploaded files should be validated carefully. File size, file type, storage location, and access permissions should be controlled. Temporary files should also be managed properly to avoid unnecessary storage and security risks.

14.8 Security and File Handling Considerations

The project combines several technologies. Python is used for machine learning and backend development. PyTorch is used to build and run the deep learning model. OpenCV is used for video and frame processing. Flask provides the backend API. HTML, CSS, and JavaScript provide the user interface.

14.7 Software and Technology Stack

The confidence score gives an indication of how strongly the model supports its selected class compared with the other available classes. It is useful for understanding the prediction, but it should not be treated as a guarantee that the prediction is correct.

14.6 Importance of Confidence Score

A model may make an incorrect prediction when two activities have similar movements, when the video quality is poor, when the person is partially hidden, or when the new video is very different from the training examples. This is why a larger and more diverse dataset can improve generalization.

14.5 Why the Model Can Make Mistakes

During training, the model receives videos together with their correct labels and learns by reducing its prediction error. During prediction, the model receives a new video without the correct label and uses the patterns learned during training to predict the most likely activity.

14.4 Difference Between Training and Prediction

The input used during prediction must be prepared in the same way as the input used during training. If the training videos are resized, normalized, and sampled in a particular way, the new video should follow the same process. This consistency helps the model interpret new inputs correctly.

14.3 Why the Same Preprocessing Is Important

The complete data flow begins when the user selects a video. The frontend sends the video to the backend. The backend prepares the video, extracts the required frames, applies the same preprocessing used during training, and sends the processed data to the model. The model produces class probabilities, the highest-probability class is selected, and the result is returned to the frontend.

14.2 End-to-End Data Flow

The system can be used as a foundation for automated video understanding. In a larger system, it could support applications such as smart surveillance, sports analysis, educational video analysis, content tagging, and activity monitoring. The current project focuses on recognizing selected activities, but the same architecture can be extended to larger real-world applications.

14.1 Practical Use of the System

Key Takeaway: The project is an end-to-end video intelligence system—not just a trained model. It connects data, deep learning, backend processing, and a user-facing application into one complete workflow.

PROJECT HIGHLIGHT

17. Conclusion

This project presents a deep learning-based Human Activity Recognition system. The system takes a video as input, extracts 16 frames, preprocesses them, and uses a 3D ResNet-18 model to learn spatial and temporal information. The trained model predicts the human activity and provides a confidence score.

The model was also integrated with a Flask backend and a web interface, creating a complete application rather than only a standalone machine learning model. The project demonstrates how deep learning can be applied to video analysis and can be further improved by increasing the dataset and adding more advanced models.

PROJECT AT A GLANCE

Project Type	Deep Learning + Computer Vision
Input	Human activity video
Frame Sampling	16 frames per video
Main Model	3D ResNet-18
Model Framework	PyTorch
Video Processing	OpenCV
Backend	Flask
Frontend	HTML, CSS, JavaScript
Output	Predicted activity + confidence score
Current Scope	5 selected activity classes

ONE-LINE PROJECT SUMMARY

A deep learning system that analyzes a video, learns visual and movement patterns using 3D ResNet-18, and predicts the human activity through a web-based application.